PROJECT REPORT

Metro Ticket Generating System in ServiceNow

A Digital Metro E-Ticketing Solution built on the ServiceNow Platform

Team ID: _____

Submitted by:Lalam Aswini

Date: 05 September 2026

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Project Overview

1.2 Purpose

2. IDEATION PHASE

2.1 Problem Statement

2.2 Empathy Map Canvas

2.3 Brainstorming

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

3.2 Solution Requirement

3.3 Data Flow Diagram

3.4 Technology Stack

4. PROJECT DESIGN

4.1 Problem Solution Fit

4.2 Proposed Solution

4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

7. RESULTS

7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

Source Code (if any)

Dataset Link

GitHub & Project Demo Link

1. INTRODUCTION

1.1 Project Overview

The Metro Ticket Generating System is a digital ticketing solution built entirely on the ServiceNow platform that enables commuters to book metro tickets online through a simple, guided Service Catalog form. Passengers select their source and destination stations, choose a single or return journey, specify the number of passengers, and select a payment mode. Based on these inputs, the system automatically calculates the applicable fare using Catalog Client Scripts and instantly generates a scannable QR-coded digital ticket that can be validated at the metro station gate. The solution eliminates manual, counter-based ticket issuance and reduces the operational load on station staff, while giving passengers a fast, contactless, and error-free booking experience.

1.2 Purpose

The purpose of this project is to digitize metro ticket booking and issuance using ServiceNow's Service Catalog, Flow Designer, Catalog Client Scripts, and Service Portal capabilities, so that:

- Commuters can book tickets anytime without visiting a physical ticket counter.
- Fare calculation is automated and error-free, based on pre-defined station-to-station pricing logic.
- Every booking is recorded digitally in ServiceNow tables for tracking, auditing, and future analytics.
- The turnaround time for ticket issuance is minimized through no-code/low-code automation.
- The system can be extended in future to support SMS/Email ticketing and integrated payment gateways.

2. IDEATION PHASE

2.1 Problem Statement

Commuters currently rely on manual, counter-based or physical ticketing systems for metro travel, which leads to long queues, delays, human errors in fare calculation, and inefficient use of station staff. There is no unified digital platform that allows passengers to book metro tickets instantly, calculate fares automatically, and receive a verifiable digital ticket without visiting a counter.

2.2 Empathy Map Canvas

Persona: Daily Metro Commuter

SAYS "I don't want to stand in a queue just to buy a ticket." "I wish I could book my ticket from my phone before I even reach the station."	**THINKS** "Will the ticket counter be crowded during peak hours?" "Is there a faster, more reliable way to pay and get my ticket?"
DOES Waits in line at the ticket counter, manually counts change, asks station staff for route and fare information, keeps a paper ticket that can get lost or torn.	**FEELS** Frustrated during peak-hour queues, anxious about missing a train while waiting, relieved and satisfied when the process is quick and digital.

2.3 Brainstorming

The following ideas were generated during the ideation phase to address the identified problem:

- Mobile/web-based digital ticket booking via the ServiceNow Service Portal.
- QR-code based digital ticket for scanning at the metro station gate.
- Automatic fare calculation based on the selected source-destination station pair.
- WhatsApp / SMS based ticket booking (identified as future scope).
- Real-time confirmation and notification once a ticket request is submitted.
- Role-based Access Control (ACL) to protect passenger and fare data.
- A future analytics dashboard for the operations team to view booking trends.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	Passenger Action	System Response	Passenger Experience
1	Discover Service	Opens Service Portal and searches / browses to 'Book A Metro Ticket'	Curious, exploring
2	Fill Booking Form	Selects starting station, destination, journey type and number of passengers	Engaged, hopeful it will be quick
3	View Fare	Fare auto-populates instantly via client script based on the selected route	Reassured, sees transparent pricing
4	Select Payment Mode	Chooses UPI, Card, or Others (with a conditional payment-mode field)	In control of the process
5	Submit & Receive Ticket	Clicks Order Now; a scannable QR ticket is generated instantly in a modal	Delighted by the instant digital ticket
6	Travel	Scans the QR ticket at the metro station gate for entry	Confident and unhindered

3.2 Solution Requirement

Software Requirements:

- ServiceNow instance (Personal Developer Instance or Enterprise Instance)
- Service Portal plugin enabled
- Flow Designer enabled
- Email Notification and Email Script support
- Scheduled Jobs capability
- Scripting support (Client Scripts, Business Rules, Script Includes)
- Modern web browser (Chrome, Edge, or Firefox)

Hardware Requirements:

- Standard desktop or laptop computer
- Minimum 4 GB RAM (8 GB recommended)
- Stable internet connection

3.3 Data Flow Diagram

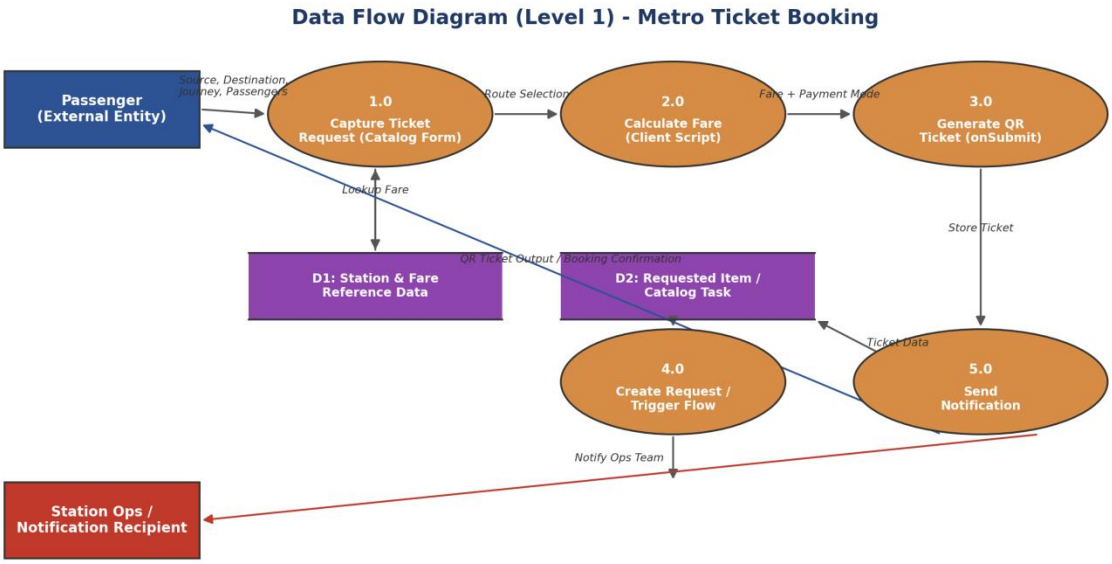


Figure: Level-1 Data Flow Diagram of the Metro Ticket Booking process

3.4 Technology Stack

Layer	Technology Used
Presentation / Frontend	ServiceNow Service Portal, Service Portal Widget (HTML + AngularJS-based server script)
Application / Logic	Catalog Client Scripts (JavaScript), Catalog UI Policies
Automation / Orchestration	Flow Designer (Request Fulfillment Automation)
Data Storage	ServiceNow Custom Tables (u_metro_station_details, u_metro_database) and platform tables (Requested Item, Catalog Task)
Security	Access Control Lists (ACLs) - role based read/write/create/delete rules
Ticket Generation	QR Server REST API (api.qrserver.com), spModal Service Portal widget
Notifications	ServiceNow Email Notifications and Email Scripts

4. PROJECT DESIGN

4.1 Problem - Solution Fit

Manual, counter-based metro ticketing creates queues, fare-calculation errors, and an inconsistent passenger experience, while station staff spend significant time on repetitive, low-value ticketing tasks. The Metro Ticket Generating System fits this problem directly: it moves the entire booking journey onto a self-service ServiceNow Service Catalog form, automates fare calculation through client scripts, and replaces the physical ticket with an instantly generated, verifiable QR code - removing the counter as a bottleneck while keeping the process fast, accurate, and auditable.

4.2 Proposed Solution

S.No.	Parameter	Description
1	Problem Statement	Commuters currently rely on manual, counter-based or physical ticketing systems for metro travel, which leads to long queues, delays, human errors in fare calculation, and inefficient use of station staff.
2	Idea / Solution Description	A "Metro Ticket Generating System" built entirely on ServiceNow, using the Service Catalog, Flow Designer, Catalog Client Scripts, and Service Portal Widgets, to auto-calculate fares and instantly generate a scannable QR digital ticket.
3	Novelty / Uniqueness	Built natively on a no-code/low-code enterprise workflow platform - combining Service Catalog automation, dynamic fare-calculation client scripts, conditional UI Policies, and real-time QR generation in one unified fulfillment workflow.
4	Social Impact / Customer Satisfaction	Reduces wait times at counters, minimizes human error in fare collection, supports environmental sustainability by cutting paper ticket usage, and encourages digital adoption among commuters.
5	Business Model (Revenue Model)	Revenue generated through digital ticket fare collection via integrated payment modes (UPI, Card, Others); extendable as a licensed digital ticketing add-on for metro operators on ServiceNow.
6	Scalability of the Solution	Can be scaled to more stations/cities by extending reference tables; future scope includes SMS/Email ticketing, real payment gateway integration, and a travel analytics dashboard.

4.3 Solution Architecture

The solution architecture bridges the business need (fast, digital, error-free metro ticketing) with a ServiceNow-native technical implementation - spanning the Service Portal (presentation), Service Catalog with Client Scripts, UI Policies and QR generation (application/logic), Flow Designer with Notifications (automation/orchestration), and custom tables secured by ACLs (data).

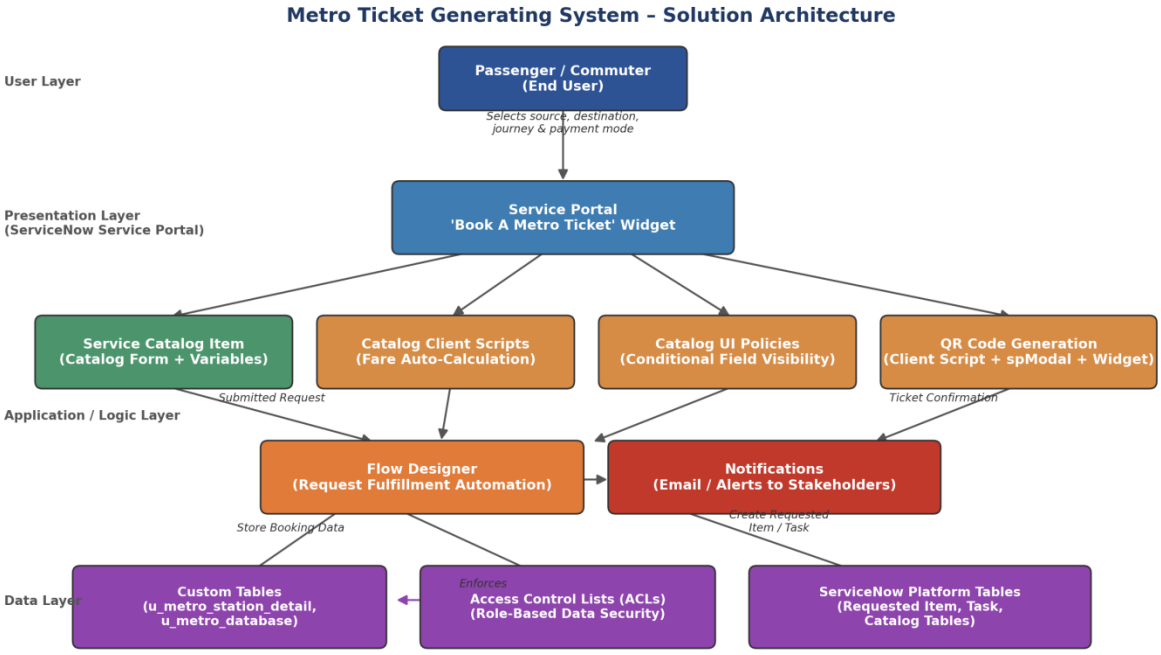


Figure: Solution architecture and data flow of the Metro Ticket Generating System in ServiceNow

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Stakeholders identified: Passengers (end users), Station Managers, Metro Operations Team, and IT Admins.

The project was executed across 5 phases, following the roadmap below:

Phase	Title	Key Activities	Progress
Phase 1	Requirement Analysis & Planning	Business Objectives, Functional Scope, Stakeholder Mapping, Execution Roadmap	50%
Phase 2	Backend Development & Configurations	Custom table creation (u_metro_station_details), field/column setup, data architecture	50%
Phase 3	UI/UX Development & Customization	Service Catalog item creation, variable configuration, Catalog UI Policies	50%
Phase 4	Data Migration, Testing & Security	Catalog Client Scripts (fare logic, QR generation), Access Control (ACLs), Data Integrity checks	33%
Phase 5	Deployment, Documentation & Final Presentation	Service Portal deployment, documentation, demo video planning	0%
-	Project Conclusion	Final review and sign-off	0%

Execution Roadmap (Milestones):

Catalog Form Setup → Fare Logic Script → QR Code Integration → Station Details Mapping → UI Pages → UAT & Deployment

Sprints were followed for each milestone (Form, Logic, QR, UAT), executed based on the 4-phase roadmap: Catalog Creation, Form Setup, Testing & Security, and Deployment.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Data integrity and functional checks performed:

- Field validations: non-empty source/destination, fare match, and approved travel date.
- Mandatory fields enforced on both client and server side to prevent incomplete submissions.
- Auto-populate logic configured for default values (e.g. single/return journey) using Catalog Client Scripts and UI Policies.
- Process Automation Engine used to auto-map fields and populate values supplied by the requester.

Debugging and issue resolution:

- Debugged QR encoding issues and fare mismatches using console logs and print statements in scripts.
- Used Flow Designer's Execution Details to trace flow steps, identify failures, and verify condition paths during approvals and notifications.
- Resolved misaligned or missing data bindings using Catalog Client Scripts for dynamic population.
- Confirmed UI Policy and visibility logic through Service Portal form tests and preview mode.

Functional Test Cases:

Test ID	Test Scenario	Expected Result	Status
TC01	Select valid source and destination stations	Fare fields auto-populate correctly based on route	Pass
TC02	Select 'Others' as Mode of Payment	'Enter payment mode' field becomes visible and mandatory	Pass
TC03	Submit a completed ticket request	QR code modal displays a scannable QR ticket image	Pass
TC04	Leave a mandatory field blank and submit	Form blocks submission and shows a validation message	Pass
TC05	Select Return Journey instead of Single Journey	Fare recalculates using the return-journey rate	Pass
TC06	Non-admin user attempts direct table access	Access restricted as per configured ACL rules	Pass

7. RESULTS

7.1 Output Screenshots

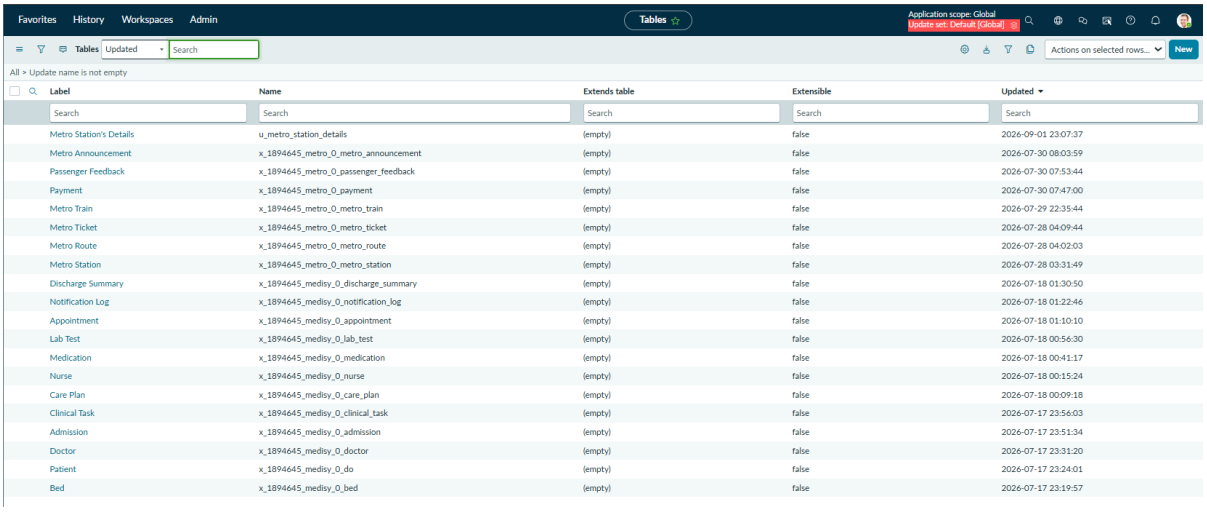


Fig 7.1: Custom Tables list in System Definition, showing the 'Metro Station's Details' (u_metro_station_details) table created for the project.

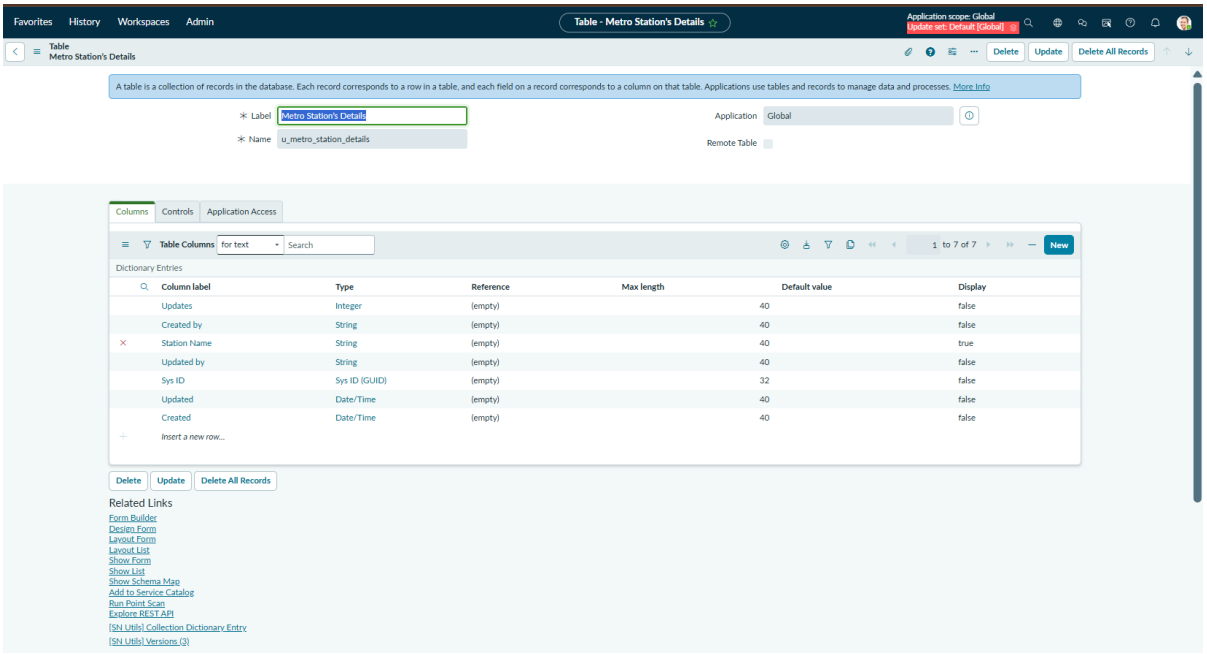


Fig 7.2: Table configuration screen for 'Metro Station's Details' showing its fields - Station Name, Sys ID, Created/Updated metadata.

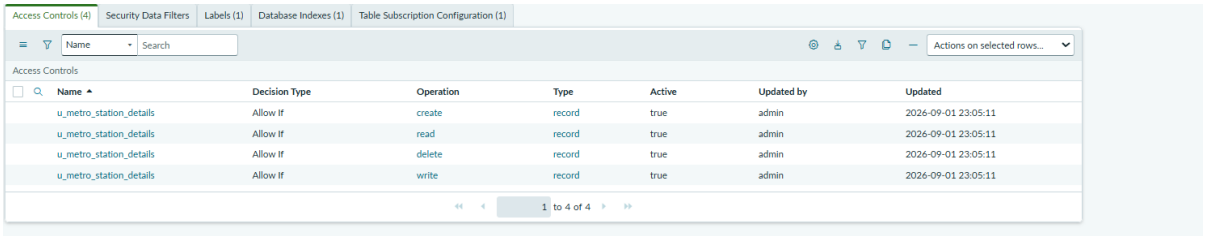


Fig 7.3: Access Controls (ACLs) configured on the u_metro_station_details table for create, read, delete, and write operations.

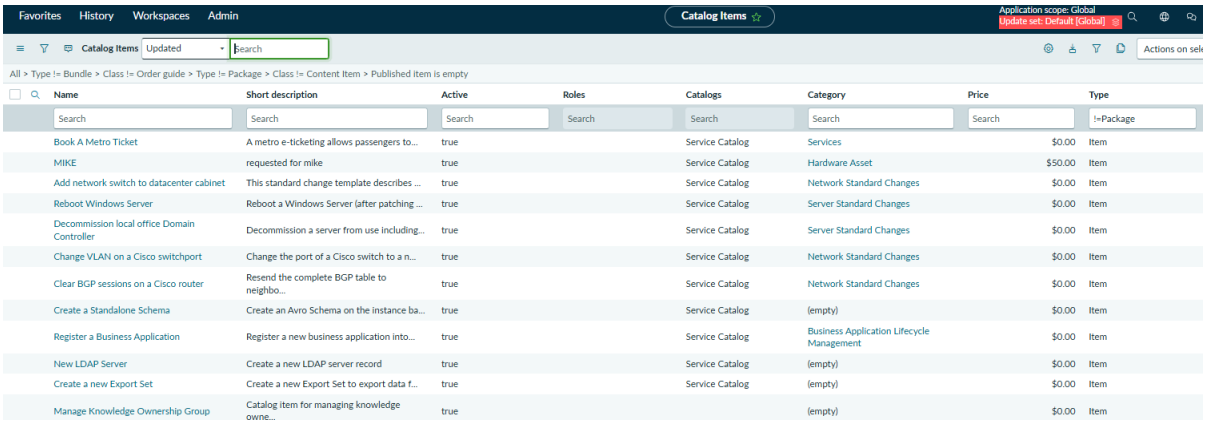


Fig 7.4: Catalog Items list showing 'Book A Metro Ticket' published under the Service Catalog, category Services.

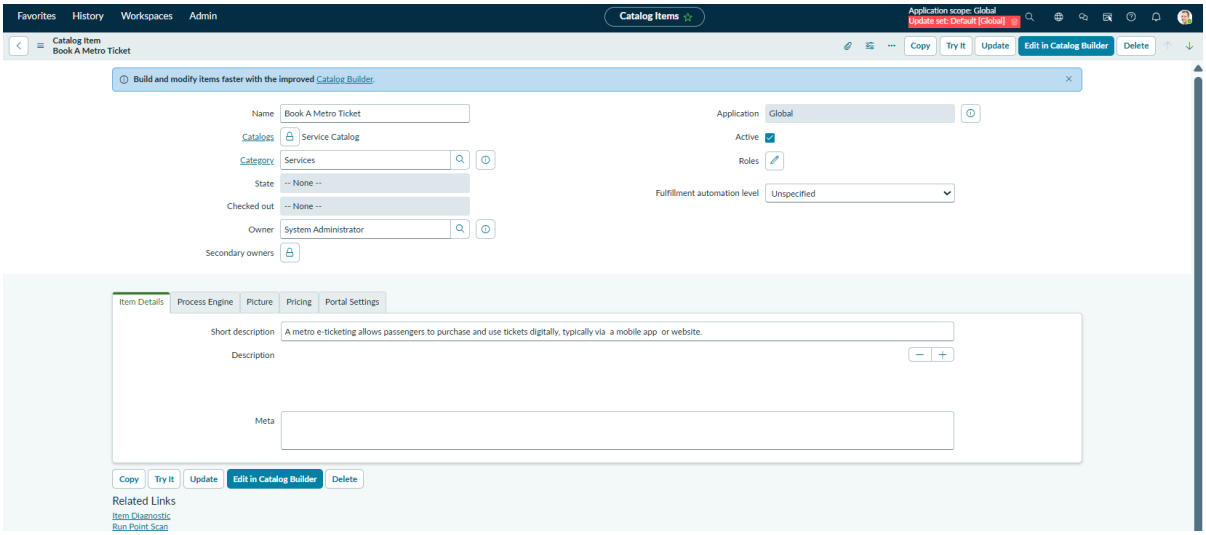


Fig 7.5: 'Book A Metro Ticket' catalog item configuration form with name, category, and short description.

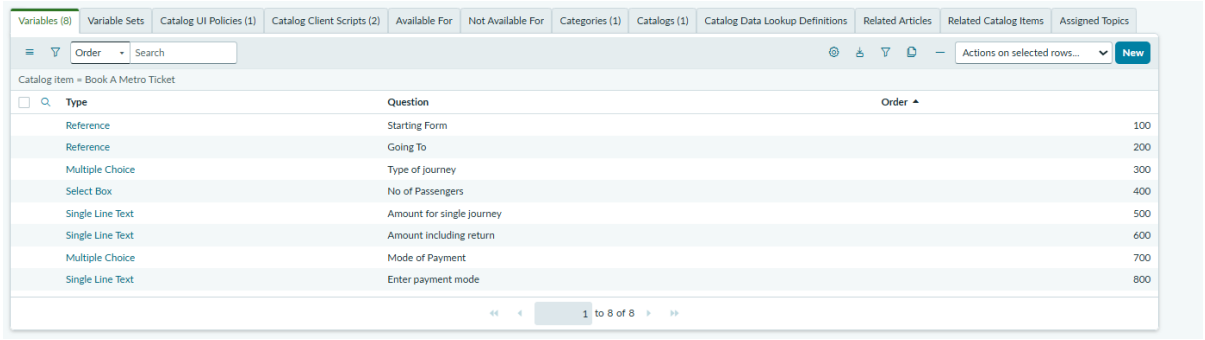


Fig 7.6: The 8 variables configured for the catalog item - Starting From, Going To, Type of Journey, No of Passengers, Amount for Single Journey, Amount Including Return, Mode of Payment, and Enter Payment Mode.

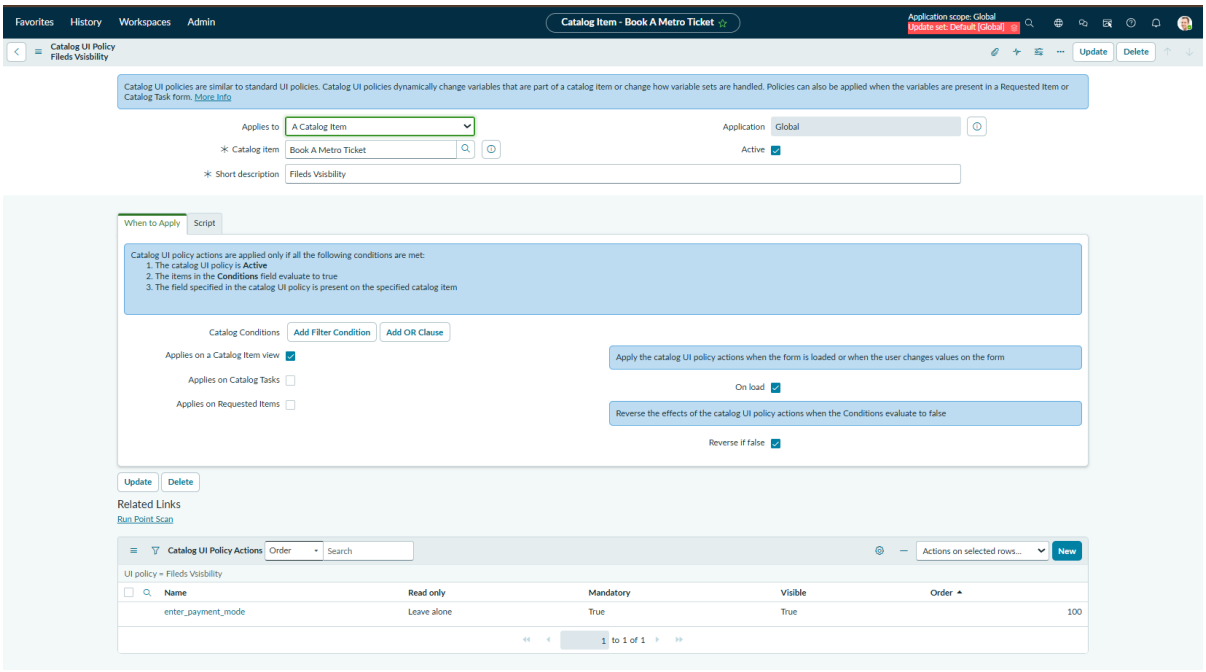


Fig 7.7: Catalog UI Policy 'Fields Visibility' - makes the 'Enter payment mode' field visible and mandatory only when Mode of Payment is 'Others'.

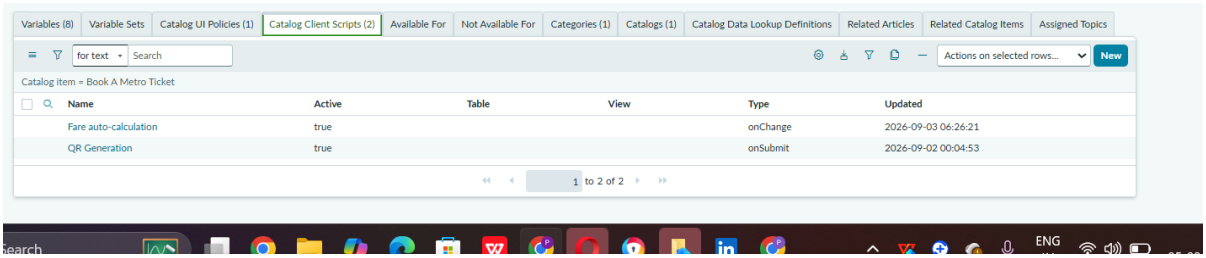


Fig 7.8: Catalog Client Scripts configured on the catalog item - 'Fare auto-calculation' (onChange) and 'QR Generation' (onSubmit).

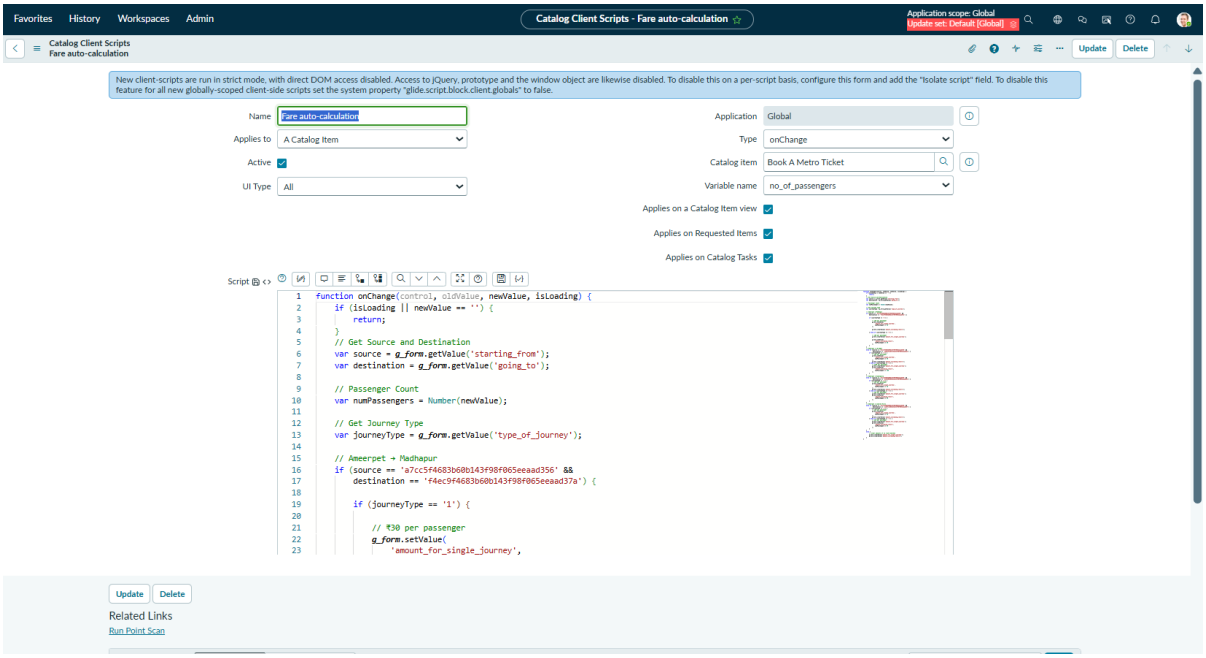


Fig 7.9: 'Fare auto-calculation' client script - onChange logic that reads source, destination, journey type and passenger count to compute the fare.

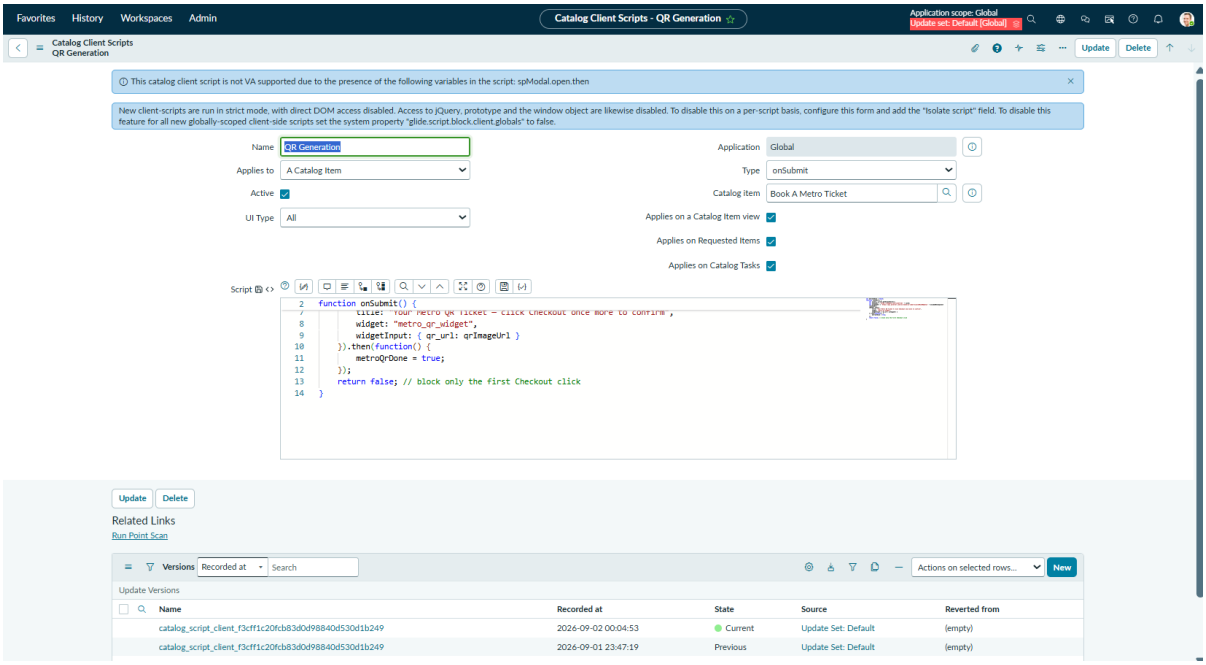


Fig 7.10: 'QR Generation' client script - onSubmit logic that builds a dummy ticket URL and opens a QR code preview using spModal.

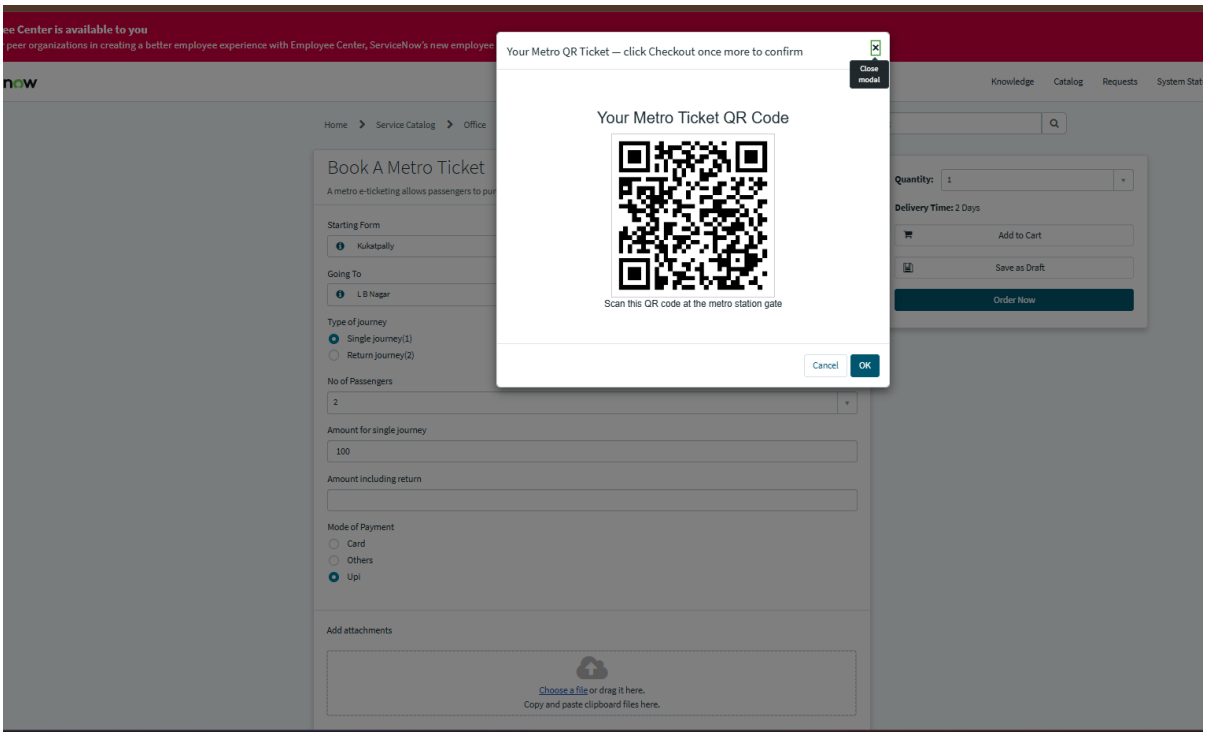


Fig 7.11: Live QR code modal shown to the passenger on the Service Portal after submitting the ticket booking form - 'Your Metro Ticket QR Code'.

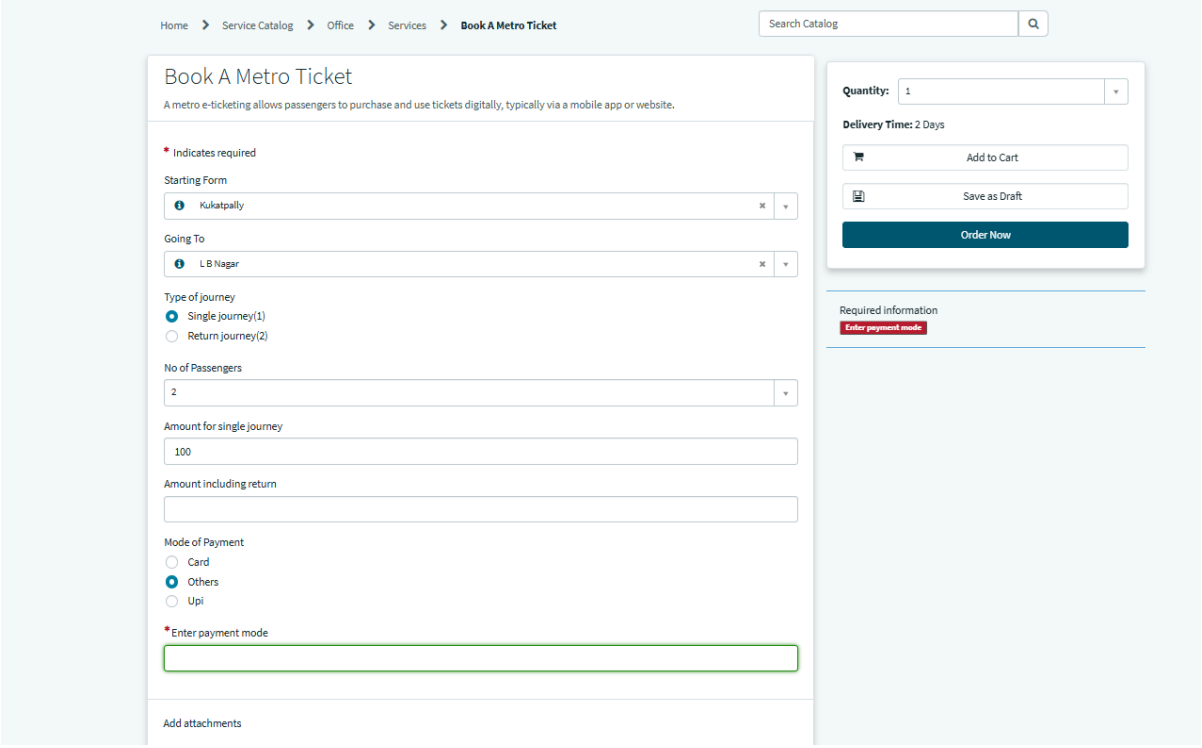


Fig 7.12: Completed 'Book A Metro Ticket' form on the Service Portal, with the conditional mandatory 'Enter payment mode' field displayed when 'Others' is selected.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Eliminates manual ticket counters and reduces passenger queues.
- Automated, error-free fare calculation based on route and passenger count.
- Digital, scannable QR ticket reduces paper usage and supports sustainability goals.
- Built on a no-code/low-code platform (ServiceNow), making it easy to maintain and extend.
- Centralized data storage in custom tables enables future reporting and analytics.
- Role-based Access Control (ACL) secures passenger and fare data.

Disadvantages

- Currently uses a dummy ticket URL and demo QR generation instead of an integrated live payment gateway.
- Requires stable internet connectivity and access to the ServiceNow Service Portal to book a ticket.
- Fares and routes are currently defined within the client script logic; adding new stations requires a manual script update rather than a fully dynamic fare engine.
- No dedicated native mobile application yet; currently accessible only via the ServiceNow Service Portal in a browser.

9. CONCLUSION

Deploying the QR-based digital ticketing capability within the Metro Ticket Generating System has strengthened the platform's real-time ticket validation process, replacing static confirmation screens with scannable, verifiable proof of booking for both commuters and station staff. By resolving Service Portal constraints through a supported spModal and widget-based architecture, the solution now renders reliably across both backend and portal experiences, ensuring a consistent outcome regardless of how users access the system. Alongside refined fare-calculation logic and cleaned-up client-side scripting, this enhancement reinforces the system's overall data integrity and operational reliability. The Metro Ticket Generating System is now well-positioned to scale toward future capabilities - such as SMS/Email-based ticketing and integrated payment gateways - while maintaining a stable, secure, and user-friendly foundation for the continued digital transformation of metro services.

10. FUTURE SCOPE

The current solution can be extended in the following directions:

- SMS-based ticketing for passengers without smartphone/app access.
- Email-based ticketing with the QR code delivered directly to the passenger's inbox.
- Integration with a real payment gateway (replacing the current dummy ticket URL and demo QR flow).
- A travel analytics dashboard for metro operators to track passenger volume, peak travel hours, and route-wise revenue.
- Extension to multiple metro lines, stations, and cities by scaling the station/fare reference tables.

11. APPENDIX

Source Code (if any)

Catalog Client Script - Fare Auto-Calculation (onChange, excerpt):

```
function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading || newValue == '') { return; }
  var source = g_form.getValue('starting_from');
  var destination = g_form.getValue('going_to');
  var numPassengers = Number(newValue);
  var journeyType = g_form.getValue('type_of_journey');

  if (source == '<Ameerpet_sys_id>' && destination == '<Madhapur_sys_id>') {
    if (journeyType == '1') {
      g_form.setValue('amount_for_single_journey', numPassengers * 30);
      g_form.clearValue('amount_including_return');
    } else if (journeyType == '2') {
      g_form.clearValue('amount_for_single_journey');
      g_form.setValue('amount_including_return', numPassengers * 60);
    }
  }
  // ... additional station-pair conditions follow the same pattern
}
```

Catalog Client Script - QR Generation (onSubmit):

```
var metroQrDone = false;
function onSubmit() {
  var sysId = g_form.getUniqueValue();
  var dummyURL = 'https://example.com/ticket?id=' + sysId;
  var qrImageUrl = 'https://api.qrserver.com/v1/create-qr-code/?size=250x250&data='
                   + encodeURIComponent(dummyURL);
  spModal.open({
    title: "Your Metro QR Ticket - click Checkout once more to confirm",
    widget: "metro_qr_widget",
    widgetInput: { qr_url: qrImageUrl }
  }).then(function() { metroQrDone = true; });
  return false; // block only the first Checkout click
}
```

Service Portal Widget - HTML Template (Metro QR Widget):

```
<div style="text-align:center; padding:25px; font-family:Arial, sans-serif;">
  <h3>Your Metro Ticket QR Code</h3>
  <img ng-src="{{data.qr_url}}" style="width:250px; height:250px; border:1px solid #ccc; padding:10px;" />
  <p>Scan this QR code at the metro station gate</p>
</div>
```

Service Portal Widget - Server Script:

```
(function() {
  data.qr_url = input ? input.qr_url : '';
})();
```

Dataset Link

Not applicable - the project uses ServiceNow custom tables (u_metro_station_details) populated directly within the instance rather than an external dataset.

GitHub & Project Demo Link

GitHub Repository: _____

Project Demo Video: _____